

TAR UMT RESORTS

AVL / ECB

保姆级开发指南

Rotation 时机与完整追踪 · K,V 资料模型 · JSON / TXT 持久化 · DAO / Interface · VIP 插队 ·
Login / Walk-In · 积分 · 多 ADT 整合

基于当前 TARUMTResortsAVL 骨架的架构审查版

2026-07-21 · Asia/Kuala_Lumpur

目录

章节	内容
0-2	结论、作业硬要求、清理后的 folder
3-4	ECB 全架构、ADT / Interface / Implementation、T 与 K,V
5-6	Rotation 时机、LL/RR/LR/RL、删除旋转、63 步大型追踪
7	TXT/CSV/JSON、DAO、大型 synthetic dataset
8-10	Member/Booking/VIP key、login/walk-in、积分与流水
11-14	AVL + Queue/Stack/List、module 文件、DAO、reports
15-16	运行验证、架构审查、最终开发顺序
Appendix	快速口试问答与交付内容

0. 先给结论：你现在应该怎样理解这个项目

当前真实状态

这个 folder 不是完成版 resort system。真正保留的是通用 key-value AVL、interface、Visitor 和一个 Hello World 主菜单。AVL 在精简前通过了 19 组 regression/invariant tests，但 test source 已依本次要求移除。Member、Booking、Room、DAO、Queue、Stack、login、VIP allocation 与 points 都还没有落地。本文后半部是下一阶段的建议架构，不代表代码已经存在。

最稳的路线是：保留当前 AVL 当团队 main ADT；每个 module 都至少实际调用它；只有业务天然需要 FIFO、undo 或临时报表时，才加入 Queue、Stack 或自写 List。不要把所有资料硬塞进同一棵 tree，也不要把业务 method 写进 AVLTree。

问题	一句话答案
1003{name,age,gender} 怎样存？	key = "1003"; value = new Member("1003", name, age, gender, ...)。AVL compare 的是 key。
Diamond=5、31425 怎样用？	5 是 tierRank；31425 是 arrivalSequence。两者与 confirmationNo 组成 VipPriorityKey，不能黏成一个神秘数字。
看到 confirmationNo 就知道 VIP？	不要。confirmationNo 只做稳定唯一 booking ID；VIP 来源是 Member/tier 或 Booking 的 tier snapshot。
需要 login 吗？	作业 prototype 不一定需要。Front desk 模式可用 confirmationNo + memberId + 身份核验；只报名字不安全。
JSON 放哪里？	data/resort-seed.json；由 dao 层 load/save。当前项目尚无 JSON loader。
换其他 ADT 要改 main AVL 吗？	不用。辅助 ADT 有自己的 interface/implementation；Control 同时持有 tree + queue/stack 即可。

1. 作业硬要求与当前代码审查

1.1 不能忽略的正式要求

- 整体 prototype 要体现 Linear ADT、Non-Linear ADT，以及明确的 searching 与自行实现 sorting。
- 团队只提交并评核 ONE collection ADT，不代表系统只能使用一种 ADT。Q&A 明确鼓励所有成员使用团队 ADT，并对 implementation/add-on method 有贡献。
- 禁止在学生代码中使用 Java Collections Framework 的 collection class/interface，例如 ArrayList、HashMap、PriorityQueue、TreeMap。Comparator 与 Iterator 可以；Collections.sort() 不可以。
- 必须遵守 ECB：Actor 只接触 Boundary；Boundary 只接触 Actor 与 Control；Control 可接触 Boundary、Entity 和其他 Control；Entity 只认识 Entity。
- 每人完成一个不重复 module；每人至少两份 report，并结合 searching、自己写的 sorting、多条件 filter。
- 最终 NetBeans project 要附实际读取的数据文件与 ReadMe.txt。数据库不是必需。
- 本评核是 Yellow / Limited AI。AI 可解释概念与检查 syntax，但不应代写完整 application 或 primary module；提交前要按 tutor 口径披露并确保每人能独立解释。

日期提醒

从现有 specification 读取到的 final deadline 是 2026-08-21 23:59，Week 11-12 每位成员 demo。若 tutor 后来更新过日期，以 tutor 最新通知为准。

1.2 当前已实现与未实现

状态	内容	判断
已实现	SearchTreeInterface<K,V>, AVLTree<K,V>, Visitor<K,V>	通用 AVL foundation ; insert/search/replace/delete/min/max/3 traversals/clear。
已验证后精简	test source 已移除	精简前 19 类 regression/invariant tests 全过 ; 含四种插入与删除旋转、200 key stress、fixed-seed mixed model。
仅占位	Main, MainMenuControl, MainMenuCLI	可以启动 , 但只显示欢迎与 Hello World。
空骨架	entity, dao, utility	只有 package-info.java , 没有业务 class。
未实现	Booking, Member, Room, VIP, loyalty, login, file loader	必须由团队下一阶段设计与实现。

2. Folder 清理后 , 哪些文件为什么保留

```
dasavljava/
├── .gitignore
├── output/pdf/
│   └── TARUMT_Resorts_AVL_ECB_Baomu_Guideline_CN.pdf
├── TARUMTResortsAVL/
│   ├── build.xml, manifest.mf, nbproject/      # NetBeans / Ant
│   ├── ReadMe.txt                             # specification
│   ├── scripts/                               # terminal compile/run
│   ├── data/
│   │   └── resort-seed.json                    #
│   ├── src/
│   │   ├── main/                             #
│   │   ├── adt/                             # shared ADT
│   │   ├── boundary/                         # console I/O
│   │   ├── control/                         # use-case orchestration
│   │   ├── entity/                          # POJO records
│   │   ├── dao/                             # persistence / seed loading
│   │   └── utility/                          # static shared helpers
```

保留 nbproject 不是留垃圾 : 它使 NetBeans 能直接 Open Project。只删除其中 private/ 本机设置。保留 ReadMe.txt 也不是多余文档 , 而是 specification 明文要求的提交物。build/、dist/、.class、.jar 都能重新产生 , 所以删除。

已判定可删	原因
所有旧 PDF、lecture slides、assignment guide copies	最终只留本指南 PDF ; 旧资料已被浓缩进本文件。
Sample Code/ 与 ECBDemo 2/	只是 reference project , 不是主项目运行依赖。
所有 .zip 备份	重复、容易交错版本 ; 真正源码已在 Git worktree。
build/, dist/, .class, .jar	generated artifacts , 可由 scripts/Ant 重建。
.DS_Store、nbproject/private	macOS/NetBeans 本机缓存。
旧 docs/*.md、root README.md、.github/modernize	不是主项目运行或当前交付所需。

3. ECB 全架构：每一层可以接谁

```

Actor (staff / guest)
  |
  v
Boundary <-----> Control <-----> other Control
| input/output only          | \
|                             | \----> Entity
|                             |
X no direct ADT              +-----> ADT interface -> implementation
X no direct DAO              |
                             +-----> DAO -> data file

Entity -> may know other Entity only
ADT    -> generic; knows no Booking/Member/Room
DAO    -> persistence detail; no menu and no rotation

```

Package	应该放	绝对不要放
main	Main.java ; 组装顶层 object 并 run	menu loop、booking rule、DAO parsing
boundary	Scanner、showMenu、readXxx、displayXxx	AVL insert/delete、points formula、file I/O
control	use-case 流程、验证、跨 tree/queue 协调	Node rotation、纯输入输出、raw JSON parsing
entity	Booking/Member/Room/PointTransaction 等 POJO	Scanner、System.out、ADT、DAO、复杂流程
adt	generic interface + implementation + iterator/visitor	VIP/room/booking 业务字眼与 policy
dao	load/save/seed、file path、格式转换	menu、tier policy、AVL internals
utility	多个 module 共用的 static sort/input/date helper	有状态 repository、module-specific business rule

3.1 一次操作怎样从 Boundary 跑到资料

```

Guest gives confirmation no.
-> FrontDeskUI.readConfirmationNo()
-> FrontDeskControl.checkIn(confirmationNo)
-> bookingTree.search(confirmationNo)
-> memberTree.search(booking.getMemberId())      # if member booking
-> roomTree.search(booking.getRoomNo())
-> Control validates status and identity
-> Booking.setStatus(CHECKED_IN)
-> Room.setStatus(OCCUPIED)
-> BookingDAO / aggregate DataStore save
-> FrontDeskUI.displayCheckInResult(...)

Boundary never touches tree. Entity never reads Scanner. AVL never prints.

```

3.2 Composition root：共享资料只 new 一次

真正整合时，建议在 MainMenuControl 或独立 ApplicationContext 建立共享 collection，再 constructor injection 给各 module control。不要每个 Control 都 new 自己的 bookingTree；否则 Walk-In 新增的 booking，Front Desk 会完全看不到。

```

SearchTreeInterface<String, Booking> bookingTree = new AVLTree<>();
SearchTreeInterface<String, Member> memberTree  = new AVLTree<>();
SearchTreeInterface<String, Room> roomTree       = new AVLTree<>();

WalkInControl walkIn = new WalkInControl(bookingTree, standardQueue, ...);
VipControl vip      = new VipControl(memberTree, bookingTree, vipTree, ...);
FrontDeskControl fd  = new FrontDeskControl(bookingTree, memberTree, roomTree, ...);

// Control      object reference      copy

```

4. ADT、Interface 与 Implementation 到底是什么

4.1 ADT 是行为合约，不是某个 class 名

ADT (Abstract Data Type) 先定义外部可以做什么，例如 insert、search、delete、traverse；implementation 决定内部怎样做到。SearchTreeInterface 是 contract，AVLTree 是一种会自平衡的 implementation。Control 应声明 interface 类型，让实现可以替换，也限制它只能调用获准的公共 method。

概念	本项目例子	职责
ADT specification	报告中的 Search Tree 操作/前后条件	语言无关地定义行为。
Java interface	SearchTreeInterface<K,V>	把合约变成编译器能检查的 method signatures。
implementation	AVLTree<K,V>	Node、height、rotation、recursion 的实际代码。
client	Module Control	只通过 interface 调用，不接触 Node。
callback	Visitor<K,V>	Control 决定每个 traversal entry 要打印、筛选或统计什么。

为什么 Node 必须 private

如果 Control 能 setLeft、setRight 或改 height，就可以绕过 rebalance 破坏整棵树。Node 是 implementation detail；对外只返回 key/value 或通过 Visitor/Iterator 遍历。

4.2 <T> 与 <K,V> 的差别

写法	存什么	比较谁	适合
AVLTree<T>	一个 T entry	T.compareTo(other)	entry 本身就是排序单位，例如 Student 按 studentId 比。
AVLTree<K,V>	一对 key + value	只比较 K；V 不必 Comparable	用 ID 查完整 object，例如 memberId -> Member。
Map<K,V> 观念	key-value pair	按 key 定位	概念相近，但本作业不能直接用 Java TreeMap/HashMap。

```
// Single generic entry
AVLTree<Member> tree;
// Member must implement Comparable<Member>; one ordering is baked into Member.

// Key-value version used by current project
SearchTreeInterface<String, Member> memberTree = new AVLTree<>();
memberTree.insert("1003", new Member("1003", "Ali", 21, "MALE"));

// Tree compares only "1003". It stores and returns the entire Member as value.
```

所以 `1003{name,age,gender}` 不是一个 Java storage syntax。它只是人类画图的 shorthand。正确 object 是 Member；正确 AVL entry 是 `("1003", memberObject)`。Node 内部实际保存 `key, value, left, right, height`。

最重要的 key 规则

key 入树后，任何参与 compareTo 的字段都不可原地修改。member name、age、points 是 value field，可更新；memberId 若是 key，要改就 delete(oldId) 再 insert(newId, value)。VipPriorityKey 的 tier/sequence 也一样。

4.3 新增一个 ADT operation 会动到哪里

Step	文件	必须做什么
1	ADT specification in report	写 generic 行为、precondition、postcondition、return、empty case。
2	src/adt/SearchTreeInterface.java	加入 generic signature 与 Javadoc，不出现 VIP/Booking 字眼。
3	src/adt/AVLTree.java	public wrapper + private recursive helper；复用 rebalance。
4	开发时的临时验证	normal/empty/duplicate/连续操作/rotation/invariant；确认后可不放进精简交付结构。
5	对应 Control	仍声明 interface 类型并调用新 method。
6	ReadMe/report/demo	记录 contract version、作者贡献、使用场景。

5. AVL rotation : 什么时候发生 , 什么时候绝对不发生

5.1 高度与 balance factor

```
height(null) = 0
height(leaf) = 1
height(node) = 1 + max(height(left), height(right))
balanceFactor(node) = height(left) - height(right)

legal: -1, 0, +1
rotate only when balanceFactor > +1 or balanceFactor < -1
```

操作	会否 rotation	原因
insert new key	可能	递归回程更新每个 ancestor height ; 首个 abs(bf)>1 的 subtree 修复。
delete existing key	可能 , 而且可多层	子树变矮后多个 ancestor 都可能失衡 , 必须一路 rebalance 到 root。
replace same key's value	不会	key 与结构不变 , 高度不变。
search/contains/traversal	不会	read-only , 不改结构。
insert duplicate rejected	不会	没有新 Node , 结构与 height 都不变。
modify a value field	不会	前提是该 field 不参与 key compareTo。
change a key field	禁止原地改	必须 delete old + insert new ; 两个动作分别可能 rotation。

Rotation 的真正时机

不是每 insert 一次就 rotate , 也不是等整棵树最后再看。insert/delete 递归走到底后 , 在 return 的回程逐层 updateHeight -> compute bf -> rebalance。rotation 返回新的 subtree root , parent 和 public root 都必须接住 return value。

5.2 四种 case 的判断表

Case	失衡 node	child 方向	动作
LL	bf > 1	left child bf >= 0	rotateRight(node)
RR	bf < -1	right child bf <= 0	rotateLeft(node)
LR	bf > 1	left child bf < 0	node.left=rotateLeft(node.left); rotateRight(node)
RL	bf < -1	right child bf > 0	node.right=rotateRight(node.right); rotateLeft(node)

5.3 LL : 30, 20, 10 的完整步骤

```
Step 1 insert 30:      30(h=1,bf=0)

Step 2 insert 20:      30(h=2,bf=+1)
                       /
                       20(h=1)

Step 3 insert 10:      30(h=3,bf=+2) &lt;- first unbalanced pivot
                       /
                       20(h=2,bf=+1)
                      /
                     10

LL because pivot bf>1 and left child bf>=0.
rotateRight(30):

                20(h=2,bf=0)
               /
              10    30
```


指针顺序：newRoot=30.left(20)；transfer=20.right(null)；20.right=30；30.left=transfer；先更新 30 height，再更新 20 height。

5.4 RR : 10, 20, 30 的完整步骤

```
Before fix:          10(h=3,bf=-2) &lt;- pivot
                      /
                     20(h=2,bf=-1)
                      /
                     30

RR because pivot bf<-1 and right child bf<=0.
rotateLeft(10):

          20(h=2,bf=0)
         /
        10      30
```

5.5 LR : 30, 10, 20 的完整步骤

```
Before fix:          30(h=3,bf=+2) &lt;- pivot
                      /
                     10(h=2,bf=-1)
                      /
                     20

LR is a bent shape. One rotation is not enough.

1) pivot.left = rotateLeft(10)    -&gt;      30
                                     /
                                    20
                                     /
                                    10

2) rotateRight(30)                -&gt;      20
                                     /
                                    10      30
```

5.6 RL : 10, 30, 20 的完整步骤

```
Before fix:          10(h=3,bf=-2) &lt;- pivot
                      /
                     20
                      /
                     30(h=2,bf=+1)

1) pivot.right = rotateRight(30)
2) rotateLeft(10)

After fix:           20
                     /
                    10      30
```

5.7 删除为什么可能连续旋转

```

Deletion LL example used during validation:
insert: 30,20,40,10,25

      30
     /  \
    /    \ 20  40
   /      \
  10      25

      delete 40 -> pivot 30 bf==+2
      left child 20 bf=0 => LL rule

rotateRight(30):
      20
     /  \
    /    \ 10  30
   /      \
  25

Expected preorder: 20,10,30,25

```

插入在 AVL 中通常修复首个失衡点后，上层高度会稳定；删除可能让 subtree 高度继续下降，所以每个 ancestor 都要 rebalance。当前实现的 rebalance 使用 child bf 判断，insert/delete 可以共用，也覆盖 child bf=0 的删除边界。

6. 大型 rotation 数据集：63 次 insert 的完整追踪

下面用固定 seed 2063 将 key 1001..1063 打乱，逐一插入空 AVL。这个顺序故意覆盖四种 rotation。表中 pivot 是回程时第一个 $\text{abs}(\text{bf}) > 1$ 的 node；overall root 是该步完成后整棵树 root。

结果摘要

63 insert；LL=2、RR=10、LR=8、RL=7；最终 root=1034、height=7。理想满树 63 nodes 的最低 height 是 6；随机顺序得到 7 仍满足 AVL。

6.1 完整 insertion order

```
1048, 1006, 1034, 1026, 1017, 1036, 1011, 1028, 1003
1018, 1004, 1039, 1037, 1045, 1002, 1008, 1047, 1059
1030, 1053, 1009, 1050, 1019, 1022, 1055, 1001, 1060
1057, 1035, 1046, 1013, 1051, 1023, 1041, 1025, 1038
1040, 1056, 1027, 1031, 1042, 1052, 1020, 1043, 1029
1016, 1062, 1044, 1021, 1032, 1014, 1005, 1015, 1007
1058, 1049, 1024, 1063, 1033, 1012, 1010, 1061, 1054
```

6.2 每一步是否 rotate

Step	Insert	结果	primitive rotation	Root	H
1	1048	balanced	none	1048	1
2	1006	balanced	none	1048	2
3	1034	pivot=1048, LR	rotateLeft(left of 1048); rotateRight(1048)	1034	2
4	1026	balanced	none	1034	3
5	1017	pivot=1006, RL	rotateRight(right of 1006); rotateLeft(1006)	1034	3
6	1036	balanced	none	1034	3
7	1011	balanced	none	1034	4
8	1028	balanced	none	1034	4
9	1003	balanced	none	1034	4
10	1018	balanced	none	1034	4
11	1004	pivot=1034, LL	rotateRight(1034)	1017	4
12	1039	pivot=1048, LR	rotateLeft(left of 1048); rotateRight(1048)	1017	4
13	1037	balanced	none	1017	5
14	1045	balanced	none	1017	5
15	1002	balanced	none	1017	5
16	1008	balanced	none	1017	5
17	1047	pivot=1048, LR	rotateLeft(left of 1048); rotateRight(1048)	1017	5
18	1059	pivot=1034, RR	rotateLeft(1034)	1017	5
19	1030	pivot=1017, RL	rotateRight(right of 1017); rotateLeft(1017)	1034	5
20	1053	pivot=1048, RL	rotateRight(right of 1048); rotateLeft(1048)	1034	5
21	1009	pivot=1011, LR	rotateLeft(left of 1011); rotateRight(1011)	1034	5
22	1050	pivot=1047, RL	rotateRight(right of 1047); rotateLeft(1047)	1034	5
23	1019	balanced	none	1034	5
24	1022	pivot=1018, RR	rotateLeft(1018)	1034	5
25	1055	pivot=1039, RR	rotateLeft(1039)	1034	5
26	1001	balanced	none	1034	6
27	1060	balanced	none	1034	6
28	1057	pivot=1053, RL	rotateRight(right of 1053); rotateLeft(1053)	1034	6
29	1035	balanced	none	1034	6
30	1046	pivot=1047, LR	rotateLeft(left of 1047); rotateRight(1047)	1034	6
31	1013	balanced	none	1034	6
32	1051	pivot=1053, LR	rotateLeft(left of 1053); rotateRight(1053)	1034	6
33	1023	balanced	none	1034	6
34	1041	balanced	none	1034	6
35	1025	pivot=1022, RR	rotateLeft(1022)	1034	6
36	1038	balanced	none	1034	6
37	1040	pivot=1045, LL	rotateRight(1045)	1034	6
38	1056	balanced	none	1034	6
39	1027	balanced	none	1034	6
40	1031	balanced	none	1034	6

Step	Insert	结果	primitive rotation	Root	H
41	1042	pivot=1046, LR	rotateLeft(left of 1046); rotateRight(1046)	1034	6
42	1052	balanced	none	1034	6
43	1020	pivot=1019, RL	rotateRight(right of 1019); rotateLeft(1019)	1034	6
44	1043	balanced	none	1034	7
45	1029	balanced	none	1034	7
46	1016	pivot=1011, RR	rotateLeft(1011)	1034	7
47	1062	balanced	none	1034	7
48	1044	pivot=1042, RR	rotateLeft(1042)	1034	7
49	1021	balanced	none	1034	7
50	1032	pivot=1028, RR	rotateLeft(1028)	1034	7
51	1014	pivot=1009, RR	rotateLeft(1009)	1034	7
52	1005	balanced	none	1034	7
53	1015	pivot=1016, LR	rotateLeft(left of 1016); rotateRight(1016)	1034	7
54	1007	balanced	none	1034	7
55	1058	balanced	none	1034	7
56	1049	balanced	none	1034	7
57	1024	pivot=1023, RL	rotateRight(right of 1023); rotateLeft(1023)	1034	7
58	1063	pivot=1060, RR	rotateLeft(1060)	1034	7
59	1033	pivot=1031, RR	rotateLeft(1031)	1034	7
60	1012	balanced	none	1034	7
61	1010	balanced	none	1034	7
62	1061	balanced	none	1034	7
63	1054	balanced	none	1034	7

怎样自己核对任何一步

沿 insert path 从新 leaf 回到 root : 对每个 ancestor 重算 leftHeight/rightHeight ; bf 在 -1..1 就继续上层 ; 超出时看重 child 的 bf 判 LL/RR/LR/RL , 完成 rotation 后先更新下降 node , 再更新新 root。不要凭 key 大小猜 case。

7. Data 到底存在哪里：内存、TXT/CSV、JSON 的分工

7.1 运行中的资料与硬盘资料不是同一件事

阶段	形态	例子
硬盘	JSON/TXT/CSV record	{"memberId":"M001003", ...}
load	DAO parse + create Entity	new Member(...)
内存	AVL Node 保存 K,V reference	"M001003" -> Member object
业务更新	Control 修改 Entity / ADT	member.addPoints(500)
save	DAO serialize snapshot	写回临时文件，再安全替换正式文件

7.2 TXT/CSV 与 JSON 怎样选

格式	优点	缺点	本作业建议
pipe TXT	纯 JDK 易写；不需外部 library	要自行 escape delimiter；nested data 较麻烦	最稳，适合每 entity 一个 file。
CSV	可用 spreadsheet 看；简单 tabular	逗号/quote/newline escaping 易写错；nested data 弱	可以，但要严格 parser。
JSON	结构清楚、nested object、schema 可读	JDK 14 没内建通用 JSON parser；需 Gson/Jackson 或自己 parser	可用，但先问 tutor 外部 dependency/collection 限制。
Java serialization .dat	写 object 快	难读、版本脆弱、安全与兼容性差	不推荐做长期格式。

我的推荐

若目标是稳稳交作业：用 pipe-delimited TXT + 自己 DAO。若团队熟悉 dependency 管理且 tutor 允许：用一个 aggregate JSON snapshot。不要同时维护 members.json、bookings.json、points.json 而没有 transaction strategy，否则 save 到一半失败会出现跨文件不一致。

7.3 本次给你的大型 JSON

路径	内容	状态
TARUMTResortsAVL/data/resort-seed.json	1,200 members；240 rooms；2,400 bookings；3,600 pointTransactions	已生成；synthetic=true；没有真实个人资料。

这个 file 是开发/报表/AVL 压力测试用 seed，不代表当前程序已读取它。实现 loader 前，run.sh 仍只显示 Hello World。JSON 中 confirmationNo 固定为 8 位 String，前导 0 不会丢；Member points 与三笔 transaction delta 对得上。

```
{
  &quot;schemaVersion&quot;: 1,
  &quot;synthetic&quot;: true,
  &quot;members&quot;: [
    {
      &quot;memberId&quot;: &quot;M001003&quot;,
      &quot;name&quot;: &quot;Synthetic Guest 0003&quot;,
      &quot;age&quot;: 39,
      &quot;gender&quot;: &quot;UNDISCLOSED&quot;,
      &quot;points&quot;: 3591,
      &quot;tier&quot;: &quot;GOLD&quot;,
      &quot;tierRank&quot;: 2
    }
  ],
  &quot;bookings&quot;: [
    {
      &quot;confirmationNo&quot;: &quot;00000001&quot;,
      &quot;memberId&quot;: &quot;M001018&quot;,
      &quot;arrivalSequence&quot;: 31426,
      &quot;tierRankAtRequest&quot;: 4
    }
  ]
}
```

7.4 如果用 JSON , 要新增哪些 file

```
TARUMTResortsAVL/
├── data/resort-seed.json
├── lib/gson-x.y.z.jar           # only if tutor approves
├── src/dao/JsonDataStore.java   # parse/write; no business rules
├── src/dao/DataSnapshot.java    # arrays: Member[], Room[], Booking[]...
└── src/entity/...              # POJO fields must match JSON

NetBeans:
Project Properties -> Libraries -> Add JAR/Folder -> lib/gson-x.y.z.jar

Important: deserialize to arrays/DTOs, not java.util.List/Map in student code,
because the assignment prohibits Java Collections Framework collections.
```

- Load once at startup: `JsonDataStore.load()` -> `DataSnapshot` -> insert every array item into the proper AVL/Queue/Stack.
- Save after a successful use case or via explicit Save/Exit: traverse collections into arrays, serialize a complete snapshot.
- Write to `data/resort-seed.json.tmp` first, close/flush, then replace the main file. Do not overwrite the only good file directly.
- Keep `schemaVersion`. When fields change later, loader can detect and migrate/reject old format cleanly.
- Do not store plaintext password. If login is implemented, store `passwordHash + salt` or delegate auth; for this assignment, staff-side verification is simpler.

8. Member、Booking、VIP priority：资料到底怎样拆

8.1 建议 Entity fields

Entity	关键 fields	主 key / index
Member	memberId, name, age, gender, phoneLast4, points, tier, active	memberTree: memberId -> Member
Booking	confirmationNo, memberId?, guestNameSnapshot, roomNo, status, tierAtRequest, arrivalSequence	bookingTree: confirmationNo -> Booking
Room	roomNo, roomType, nightlyRate, status	roomTree: roomNo -> Room
PointTransaction	transactionId, memberId, bookingConfirmationNo?, type, pointsDelta, occurredAt	transactionTree: transactionId -> PointTransaction
VipPriorityKey	tierRank, arrivalSequence, confirmationNo	vipTree key ; value 指向同一 Booking
RoomStatusChange	roomNo, oldStatus, newStatus, changedAt, staffId	housekeeping Stack value

一个 Booking 可以同时出现在两棵 tree 吗？

可以，作为两个 index：bookingTree 用 confirmationNo exact search；vipTree 用 VipPriorityKey 排 priority。两个 value 可指向同一个 Booking object。完成 allocation 后从 vipTree 删除 priority entry，但 bookingTree 仍保留历史 booking。

8.2 Diamond=5 与 31425 的正确意思

```

tierRank = 5                // DIAMOND business rank
arrivalSequence = 31425     // monotonic arrival order, not points
confirmationNo = "03142567" // stable unique booking ID, String

VipPriorityKey key = new VipPriorityKey(
    tierRank, arrivalSequence, confirmationNo
);
vipTree.insert(key, booking);

```

不要存成 `531425` 再靠 substring 拆，因为位数、可读性、overflow、policy 变化都会出问题。也不要只用 tierRank 当 key：所有 Diamond 都会 duplicate。arrivalSequence 保证同 tier 的先到先服务，confirmationNo 做最终唯一 tiebreaker。

8.3 compareTo 怎样让 Diamond 先、同 tier 早到先

```

public int compareTo(VipPriorityKey other) {
    int tierCmp = Integer.compare(this.tierRank, other.tierRank);
    if (tierCmp != 0) return tierCmp;           // higher tier = larger key

    int arrivalCmp = Long.compare(
        other.arrivalSequence, this.arrivalSequence
    );
    if (arrivalCmp != 0) return arrivalCmp;     // earlier = larger key

    return this.confirmationNo.compareTo(other.confirmationNo);
}

// Highest priority is tree.getLargestKey(), the right-most key.
// It is NOT necessarily the AVL root.

```

Tier 顺序是业务假设

本指南按你的要求采用 DIAMOND=5, PLATINUM=4, ELITE=3, GOLD=2, SILVER=1。正式提交前必须让团队/tutor确认名称与高低顺序；不要只凭英文直觉。用 enum 集中定义 rank，Control 不要散落 magic number 5。

8.4 完整 VIP allocation flow

```
New request:
1. Boundary reads confirmationNo/memberId or creates new booking input.
2. Control searches memberTree.search(memberId).
3. Verify identity; reject inactive member.
4. Read Member.tierRank; copy tierAtRequest into Booking for audit.
5. bookingTree.insert(confirmationNo, booking).
6. if tierRank >= VIP threshold:
    vipTree.insert(new VipPriorityKey(rank, seq, confirmNo), booking)
    else:
        standardQueue.enqueue(booking)

When a room becomes READY:
1. if !vipTree.isEmpty():
    key = vipTree.getLargestKey()
    booking = vipTree.search(key)
    vipTree.delete(key)
    else:
        booking = standardQueue.dequeue()
2. Assign room; update Booking and Room status.
3. Keep booking in bookingTree; persist snapshot.
4. Display VIP badge from tierAtRequest/member, not from confirmationNo.
```

8.5 Confirmation number 要不要特殊编码 VIP

不建议。confirmationNo 的唯一职责是稳定、唯一、可查询。tier 会升级/降级；把 tier 编码进号码会让旧号码骗人、泄漏会员等级、增加 parser 依赖。可以在 UI 显示 `DIAMOND` badge，或在 Booking 保存 `tierAtRequest`，但 source of truth 仍是 Member 与 audit snapshot。

9. Login、Walk-In 与冒认 VIP 的处理

9.1 作业 prototype 必须 login 吗？

不是必需。题面是 console-based prototype，重点是 ADT/algorithm/ECB，不是完整 cybersecurity。最省风险的范围是 staff-operated front desk console：顾客不自己登录，staff 录入 confirmationNo/memberId 并做人身核验。

模式	顾客提供	系统核验	适合
Member booking	confirmationNo + memberId	Member active + phone last4/physical ID；booking memberId must match	已预订会员
Walk-in member	memberId + physical ID/phone last4	memberTree exact search；不要只报名字	现场会员
Walk-in non-member	姓名 + basic details	建立 Guest snapshot；tierRank=0；进 standard queue	普通 walk-in
Self-service login	memberId/email + password/OTP	hash verification/session/lockout	只有 team 明确把 auth 纳入 scope 才做

别人报 VIP 名字怎么办

名字不是 identity key，也可能重名。系统只能用 memberId/confirmationNo exact search，再比对 booking 的 memberId 与身份证明/phone last4。核验失败就不能使用 tier priority；记录 failed verification，但不要在 console 暴露完整手机号或敏感资料。

9.2 如果真的做 login

- 新增 entity/UserAccount.java：accountId、memberId、passwordHash、salt、status、failedAttempts；不要把 password 放 Member。
- 新增 control/AuthControl.java：login/logout/lockout；新增 boundary/LoginUI.java：只负责读 credential 与显示结果。
- 新增 dao/AccountDAO.java 或纳入 aggregate DataStore；不存 plaintext password。
- 登录只证明 account identity，不直接决定 priority。登录后仍从 memberTree 找 Member.tier。
- 不要用 confirmationNo 当 password；它可能印在 email/receipt，也不应泄漏 tier。

如果不做 login，README 与 report 要明确 assumption：本系统由授权 front-desk staff 操作，顾客身份由现场证件或 phone last4 核验；prototype 不提供 public self-service account access。这个范围清楚，比做半套不安全 login 更好。

10. 积分怎样记录、增加、兑换与防止重复

10.1 不要只改 Member.points 而没有流水

Member.points 是当前余额，方便 $O(\log n)$ search 后直接显示；PointTransaction 是不可变 audit ledger，解释余额为什么改变。两者要一起更新。若只存一个 points 数字，重复 award、人工 adjustment 与 redemption 都无法审计。

```
Points earn after eligible CHECKED_OUT:
1. booking = bookingTree.search(confirmationNo)
2. require booking.status == CHECKED_OUT
3. require booking.memberId != null
4. require booking.pointsAwarded == false      # idempotency guard
5. member = memberTree.search(booking.memberId)
6. earned = calculateEligibleSpend(...)        # Control business rule
7. member.addPoints(earned)
8. transactionTree.insert(txId, new PointTransaction(..., +earned))
9. booking.setPointsAwarded(true)
10. recompute tier; if changed, record/display upgrade
11. save one consistent snapshot
```

动作	pointsDelta	检查
入住完成赚积分	+earned	同一 confirmationNo 只能 award 一次。
兑换	-redeemed	balance >= redeemed ; transaction first-class。
过期	-expired	依据 expiry policy ; 保留 reference。
人工调整	+/-adjustment	必须有 reason/staffId ; 不可静默改值。

Tier 更新会不会重排正在等待的 VIP ?

建议 Booking 保存 tierAtRequest 作为公平与审计 snapshot。若 policy 要实时 tier，就必须从 vipTree delete old VipPriorityKey 再 insert new key；不能原地改 key fields。两种 policy 都可，但必须全队统一并写入 report。

10.2 Points 与 AVL index

需求	结构	原因
按 memberId 找余额	memberTree<String,Member>	exact search $O(\log n)$ 。
按 transactionId 找流水	transactionTree<String,PointTransaction>	unique audit lookup。
按 points 排名	临时 array/custom List + self-written sort	points 常变；维护第二 AVL 会有同步成本。
长期排行榜高频查询	可选 second AVL<PointsKey,Member>	每次 points 改变必须 delete old key + insert new key。

11. Main AVL 与 Queue/Stack/List 怎样一起用

一个 module 可以同时持有多个 ADT。ADT 不直接互相调用；Control 才是协调者。这样 AVL 保持 generic，Queue 保持 FIFO，Stack 保持 LIFO，业务组合留在 use-case 层。

Module	Main AVL 用法	辅助 ADT	组合逻辑
Walk-In	confirmationNo -> Booking index	Queue<Booking>	enqueue 保持 chronological；allocation dequeue；AVL 提供 search/cancel。
VIP	VipPriorityKey -> Booking；memberId -> Member	可不加；AVL max 已能 priority	先 member search，再 insert priority tree；room ready 时取 largest。
Housekeeping	roomNo -> Room current state	Stack<RoomStatusChange>	update 前 push old/new change；undo pop 再恢复 Room。
Front Desk	booking/member/room 三个共享 index	可选 Queue<ServiceRequest>	exact search 后跨 Entity 更新 check-in/out。
Loyalty	member/transaction index	custom List/array for report	search member、改 points、记 ledger、report 自写 sort。
Reports	traverse/filter candidate	custom List/array	filter -> collect -> self-written merge/quick sort -> format。

11.1 Walk-In : Queue + AVL

```

WalkInControl fields:
SearchTreeInterface<String, Booking> bookingTree; // shared search index
QueueInterface<Booking> standardQueue;           // FIFO chronology

addWalkIn():
    create Booking -> bookingTree.insert(confirmNo, booking)
    -> standardQueue.enqueue(booking)

cancel(confirmNo):
    booking = bookingTree.search(confirmNo)
    booking.setStatus(CANCELLED)
    // If Queue has no arbitrary removal, dequeue skips cancelled records later.

allocateStandardRoom():
    do booking = standardQueue.dequeue()
    while booking != null && booking.status == CANCELLED
    assign the first valid booking
  
```

11.2 Housekeeping : Stack + AVL

```

HousekeepingControl fields:
SearchTreeInterface<String, Room> roomTree;
StackInterface<RoomStatusChange> undoStack;

updateStatus(roomNo, newStatus):
    room = roomTree.search(roomNo)
    undoStack.push(new RoomStatusChange(roomNo, room.status, newStatus, ...))
    room.setStatus(newStatus)

undoLast():
    change = undoStack.pop()
    room = roomTree.search(change.roomNo)
    room.setStatus(change.oldStatus)
  
```

辅助 ADT 的 interface 也要分开

新增 Queue 时创建 QueueInterface<T> + LinkedQueue<T>；新增 Stack 时创建 StackInterface<T> + ArrayStack<T> 或 LinkedStack<T>。不要让 Queue extends SearchTreeInterface，也不要吧 enqueue/pop 塞进 AVL interface。

12. 每个 module 会动到什么 folder/file

Module	entity/	boundary/	control/	dao/ / adt/
Walk-In	Booking, GuestSnapshot	WalkInUI	WalkInControl	BookingDAO; QueueInterface + implementation
VIP	VipPriorityKey; reuse Booking/Member	VipAllocationUI	VipAllocationControl	reuse DAO; optional removeLargest add-on + tests
Housekeeping	Room, RoomStatusChange, enum status	HousekeepingUI	HousekeepingControl	RoomDAO; StackInterface + implementation
Front Desk	reuse Booking/Room/Member	FrontDeskUI	FrontDeskControl	reuse DAOs; shared AVL indexes
Loyalty	Member, PointTransaction, Tier enum	LoyaltyUI	LoyaltyControl	MemberDAO/PointTransactionDAO; report sort utility

12.1 加一个 module 的固定步骤

- 先写 use cases 与 Entity fields : 什么是 stable ID、哪些状态、哪些 business invariants。
- 确定 AVL key。要 exact search 的 stable ID 通常是 key ; 要 priority 则建 immutable composite key。
- 在 entity/ 写 POJO : constructor/getter/setter/toString>equals ; 需要当 key 的 class 实现 Comparable。
- 在 boundary/ 写 input/output method , 不 import adt/dao。
- 在 control/ 写流程并声明 interface fields , 通过 constructor 收共享 object。
- 真的要 persistence 才在 dao/ 写 load/save ; 不要为了 folder 好看造空 DAO。
- 业务天然需要新 ADT 才在 adt/ 加 interface + implementation + tests ; 不要复制 Java Collections Framework。
- 加至少 module unit/integration test ; 最后在 MainMenuControl 注册 menu option 与 dependency wiring。
- 为该 member 准备两份 report : search/filter/自写 sort/structured output。

12.2 Shared file 冲突规则

文件	风险	团队规则
SearchTreeInterface.java	改 signature 会影响所有 module	先冻结 contract ; add-on 同步 implementation/test/report。
AVLTree.java	rotation bug 全系统受影响	小 patch、单一 owner review、必须跑 full test。
Booking/Member/Room.java	多人加 field/rename	明确 Entity owner ; 不要复制 Booking2。
MainMenuControl.java	多人同时接 menu	整合负责人统一 wiring ; module 提供 constructor contract。
data schema	field 变化导致 loader 失败	schemaVersion + migration/seed regenerate ; 所有人同步。

13. DAO 到底是什么，什么时候是不需要的

DAO (Data Access Object) 把资料来源细节隔离起来。Control 只说 load/save/find seed，不应该知道每一行怎样 split 或 JSON 怎样 parse。DAO 不等于 database；读取 text/JSON 也可以有 DAO。

情况	需要 DAO 吗	做法
全部 hard-coded seed	可暂时不要	由 SeedDataFactory 建 Entity arrays；仍由 composition root insert。
启动读 file，退出写 file	需要	JsonDataStore 或每 entity DAO。
只为保留空 folder	不需要 class	package-info.java 足够。
未来 database	需要	interface 可以保留，implementation 换成 JDBC；Control 不变。

当前架构中不需要什么

现在不要先建 God-object DataRepository，里面同时放 menu、所有 tree、JSON parsing、tier rule 与 reports。也不要每个 Entity 都先造 DAO interface/implementation 而程序根本不 save。当前保留 dao/package-info.java 是合理延迟决定。

13.1 简单且一致的 persistence boundary

```
interface DataStore {
    DataSnapshot load();
    void save(DataSnapshot snapshot);
}

// Startup
snapshot = dataStore.load();
compositionRoot.rebuildCollections(snapshot);

// Save
snapshot = compositionRoot.exportSnapshot();
dataStore.save(snapshot);

// DataSnapshot uses Entity arrays, not Java collection classes.
```

这是一种建议，不是当前已实现 API。若团队以每 entity DAO 更容易分工，也可以用 MemberDAO/BookingDAO/RoomDAO；重点是 Control 不碰 raw format，跨 entity 更新要有一致 save strategy。

14. Report、Searching 与 Sorting 怎样拿分而不违规

AVL inorder 输出按 AVL key 排序，但题面要求 explicit sorting algorithm。若报告按 points、date、rate 等非 key 字段，必须 traverse/filter 后放进普通 array 或课程自写 List，再调用自己写的 Merge Sort/Quick Sort。不能只 list all，也不能 Collections.sort/Arrays.sort。

```
Report pipeline:
1. Search/range/traverse AVL for candidates
2. Apply multiple criteria filters
3. Put matches into Entity[] or custom ListInterface<Entity>
4. Self-written mergeSort/quickSort with allowed Comparator
5. Calculate summary metrics
6. Control builds report model/string
7. Boundary displays structured console report
```

Module	Report 1	Report 2
Walk-In	Date range + room type + status，按 arrival time	Cancellation/no-show summary，按 count/rate
VIP	Tier + waiting time + room type，按 priority	Allocation SLA by tier，按 average wait
Housekeeping	Status + floor + age，按 overdue minutes	Staff/day completed tasks，按 task count
Front Desk	Check-in/out by date + room type，按 time	Revenue/balance exception，按 amount
Loyalty	Active members + tier + points range，按 points	Expiring points + date + tier，按 expiry then points

15. 项目怎样运行、怎样证明没坏

```
cd /Users/wilson/Desktop/dasavljava/TARUMTResortsAVL

./scripts/run.sh
# Current skeleton only shows welcome + Hello World.

# NetBeans: File -> Open Project -> choose TARUMTResortsAVL
# Main class: main.Main
```

15.1 每次 integration 的最低检查

- 先 clean generated build/，再 compile/run；不要拿旧 .class 当成功证据。
- AVL structural change 应先做独立 regression/invariant 验证；正式精简 folder 可不保留 test source。
- load 大型 seed 后核对 member/room/booking count；confirmationNo 前导 0 仍存在。
- 做一条 end-to-end：member search -> booking -> VIP/standard selection -> room update -> points -> save -> restart -> reload。
- 故意测试 duplicate key、missing member、inactive member、cancelled queue entry、empty VIP tree、insufficient points、save failure。
- 每位 member 准备能解释的 demo path 和两份 report，代码 author/AI disclosure 与实际贡献一致。

15.2 当前架构的优点与需要改进

评审项	结论	下一步
AVL encapsulation	好：Node/rotation private，Control 面向 interface	保持，不让 business method 进入 ADT。
Correctness validation	精简前 rotation/delete/stress/invariant 全过；test source 已移除	每个 add-on 开发时仍须重新验证。
ECB skeleton	方向对，但只有 placeholder	按 module owner 新增 Entity/Boundary/Control。
Visitor API	可用，但报告要解释 public dependency	与 tutor 确认是否采用 Visitor 或 Iterator。
Persistence	尚未实现	决定 TXT 或 JSON 后只做一种 authoritative format。
Main menu	尚未实现	在 composition root 注入共享 collections。
Large data	已有 synthetic JSON；当前不加载	实现 DAO loader 后做 count/reload tests。
AI compliance	源码自述为 AI reference	不要原样当 graded core；独立完成、能解释、如实披露。

16. 最终开发顺序：照这个做就不会乱

阶段	完成条件
1. Tutor/team 冻结口径	确认 generic key-value AVL 名称、tier 顺序、四人不重复 module、AI 范围。
2. Shared contract	Interface/duplicate/null/height/successor/Visitor policy 全队一致；保留精简前 19 tests 全过的记录。
3. Entity schema	Member/Booking/Room/PointTransaction/VipPriorityKey fields 与 key 明确。
4. Module skeleton	每人自己的 Entity + Boundary + Control；constructor contract 清楚。
5. Auxiliary ADT	只按真实需求加入 Queue/Stack/List；每个都有 interface/implementation/test。
6. Composition root	共享 booking/member/room tree 只 new 一次并注入；主 menu 能进入各 module。
7. Persistence	选 TXT 或 JSON；load/save/restart 数据一致；无真实敏感数据。
8. Reports	每人两份；search + multi-filter + own sort + structured output。
9. Stress/integration	大型 seed count 正确；VIP/standard/points/undo/reload 端到端通过。
10. Submission	ReadMe、report template、source author、screenshots、AI form、demo 能独立解释。

最后一句

AVL 是共享的排序/搜索引擎，不是整个 application。Entity 负责资料，Control 负责业务组合，Boundary 负责人与系统对话，DAO 负责硬盘，Queue/Stack/List 负责 AVL 不擅长的语义。把责任守住，module 才能独立开发又顺利整合。

Appendix A. 快速口试问答

问	答
为什么 AVL 比普通 BST 稳？	每次 mutation 保证 $\text{abs}(\text{balance factor}) \leq 1$ ，因此 search/insert/delete worst-case $O(\log n)$ 。
为什么最高 VIP 不在 root？	AVL root 为平衡服务，通常接近中位数；最大 key 在最右 node。
replace 为什么不 rotate？	只换 value，不换 key/child/height。
duplicate tier 怎么办？	tier 不是完整 key；加 arrivalSequence 与 confirmationNo 形成唯一 composite key。
为什么 confirmationNo 用 String？	固定 8 位且可能有前导 0；它是 identifier，不做 arithmetic。
为什么不用 guest name 当 key？	不唯一、会变、容易冒认；用 memberId/confirmationNo。
为什么 Control 声明 interface？	限制依赖并允许替换 implementation，不接触 Node。
Queue 与 AVL 谁先？	VIP tree 非空先 highest VIP；否则 standard Queue FIFO。policy 由 Control 决定。
积分怎样防重复？	Booking.pointsAwarded/idempotency + transaction ledger + unique transaction ID。
JSON 当前能直接跑吗？	不能；seed file 已生成，但要实现 DAO loader 并加获准 JSON dependency。
为什么保留 ReadMe.txt？	正式 specification 要求 NetBeans project 附运行说明。

Appendix B. 交付内容说明

本次交付

指南：/Users/wilson/Desktop/dasavljva/output/pdf/TARUMT_Resorts_AVL_ECB_Baomu_Guideline_CN.pdf

数据：/Users/wilson/Desktop/dasavljva/TARUMTResortsAVL/data/resort-seed.json

规模：1200 members、240 rooms、2400 bookings、3600 point transactions。

本文件是概念与架构指南，不是可直接提交的完整 primary module source code。正式 business rule、tier naming、external JSON library 与 Visitor/Iterator 口径均应由团队和 tutor 最终确认。